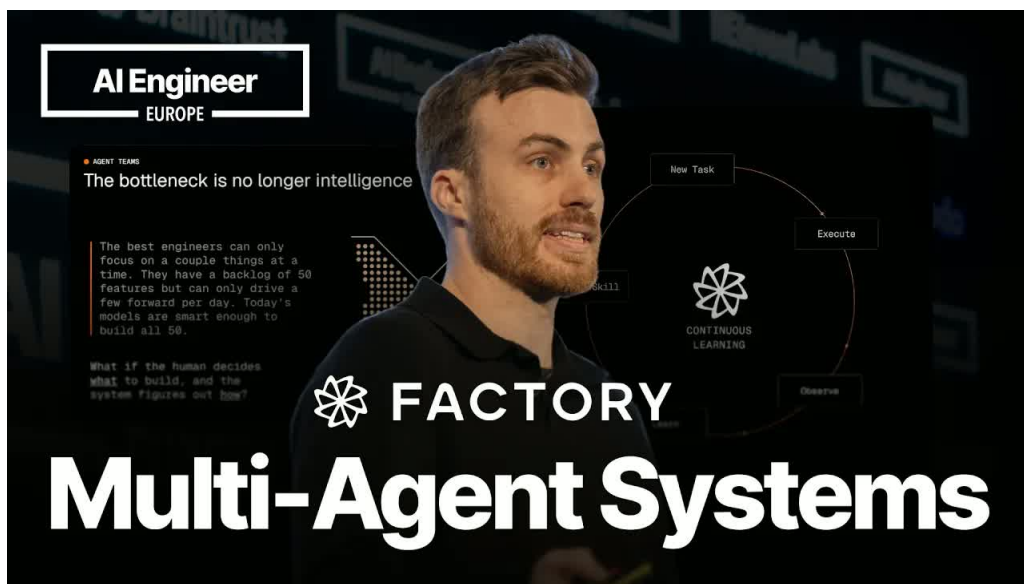


课程笔记

【AI Engineer】 | 真正能够落地交付的多智能体架构——Luke Alvoeiro (Factory)

五道口纳什 & Codex

2026 年 5 月 12 日



视频作者/频道: KrillinAI 小林

发布日期: 2026-05-10

视频时长: 18:30

视频链接: https://www.bilibili.com/video/BV1B15j62Eg1/?spm_id_from=333.1387.favlist.content.click&vd_source=6457e444c35aa58365caef6c074dd705

目录

1 从注意力瓶颈到多智能体协作积木	2
1.1 讲者背景、Goose 经验与 Factory 使命	2
1.2 软件工程的人类注意力瓶颈	3
1.3 多智能体领域的混乱与分类法需求	4
1.4 五类协作模式：从分工到共享状态	4
1.5 单一事实来源为什么决定长期连贯性	6
1.6 本章小结	6
2 Factory 任务系统与三角角色架构	7
2.1 从协作积木到任务系统	7
2.2 三角角色架构	8
2.3 结构化交接与生态视角	9
2.4 本章小结	10
3 验证合约、对抗性验证与结构化交接	10
3.1 实现后补测试的盲区	10
3.2 验证合约	11
3.3 两类验证器	12
3.4 对抗性验证	13
3.5 结构化交接	14
3.6 本章小结	15
4 运行策略、目标内并行与任务控制台	16
4.1 全局并行的诱惑	16
4.2 协调成本如何被并行放大	16
4.3 串行执行与目标内并行	17
4.4 任务控制台	18
4.5 异步执行与人在环路中	19
4.6 本章小结	20
5 模型路由、专业化智能体与生产案例	20
5.1 角色能力差异与模型路由	20
5.2 专业化智能体与机器人驯化	22
5.3 模型无关架构	22
5.4 Slack 克隆案例中的生产数据	23
5.5 企业工作流中的落地场景	24
5.6 本章小结	25
6 持续学习、软件工程经济学与开放问题	25
6.1 苦涩教训与持续学习	25
6.2 提示/技能驱动编排与硬编码状态机	26
6.3 软件工程经济学的变化	28
6.4 从原始策略到可落地系统	29
6.5 开放问题	30
6.6 本章小结	30
6.7 最终收束：把执行交给系统，把判断留给人	31

1 从注意力瓶颈到多智能体协作积木

演讲者 Luke Alvoeiro 的开场很直接：这场分享要回答一个工程问题，即怎样组建一组能够完成复杂任务的智能体团队，使它们处理的任务复杂度显著超过单个智能体的能力边界。他没有先从抽象框架讲起，而是先给出软件工程中的真实约束：模型已经足够聪明，可以同时处理许多待办事项；真正卡住交付速度的，往往是人类监督、审查、决策和交接的注意力带宽。



图 1: Luke Alvoeiro 的开场标题页：课程主题是组装能解决更复杂问题的智能体团队。¹

这也解释了多智能体系统为什么会成为软件开发工具的重要方向。若一个工程师排队了五十个功能，模型理论上可以推进其中许多项，但每个任务仍然需要范围判断、实现审查、异常裁决和结果验收。任务越多，人类越容易被上下文切换撕碎。多智能体架构的价值，正是在这种带宽缺口里出现：它试图把执行、检查、沟通和状态同步拆成可组合的协作积木，让系统承担更多“如何推进”的工作。

1.1 讲者背景、Goose 经验与 Factory 使命

演讲者来自开发工具领域。他曾在 Block 启动一个项目，后来该项目演变为 Goose，并成为开源编码智能体生态中有代表性的项目之一。随后，他在 Factory 负责核心智能体框架。按照字幕中的表述，Factory 的使命是实现软件开发生命周期的全面自主；这里的“全面自主”应被理解为让系统覆盖从需求澄清、计划执行、代码实现、质量检查到交接反馈的更多环节，而非只把某个局部动作自动化。

软件开发生命周期是整条链路

软件开发生命周期指从需求提出、范围澄清、设计、实现、测试、审查、部署到维护的连续过程。单点自动化像给流水线装上一个更快的机械臂；生命周期级自主化则要求机械臂、传送带、质检台和记录系统彼此协作。多智能体系统讨论的重点就在这里：单个动作变快之后，系统仍要处理任务分配、状态共享和质量闭环。

本章需要先固定三个基础概念。**智能体**是带有目标、上下文、工具调用能力和一定自主决策能力的软件执行单元。它像拥有工作台的工程助理：能读取任务说明，能调用代码搜索、编辑器、测试命令等工具，也能在局部目标下选择下一步动作。**多智能体系统**是多个智能体通过通信、分工、验证和共享状态共同完成任务的系统。**智能体团队**强调“团队”属性：其中的成员具有互补角色、责任边界和协作协议；成员数量增加本身没有价值，清晰的分工和可检查的交接才有价值。

¹ 视频画面时间来源：00:00:14–00:01:03；最终采用帧 00:00:18。

若把人类给出的任务记为 T ，把系统中的智能体集合记为 $\mathcal{A}$ ，一个多智能体系统可以先用最轻量形式表示为：

$$\mathcal{A} = \{a_1, a_2, \dots, a_n\}, \quad T \xrightarrow{\mathcal{A}} R$$

其中各符号含义如下：

- T ：人类希望系统完成的任务或目标，例如实现一个功能、修复一个缺陷、迁移一组接口。
- $\mathcal{A}$ ：智能体集合，也就是参与协作的全部执行单元。
- a_i ：第 i 个智能体，可对应某个局部职责，例如探索方案、编写代码或检查结果。
- n ：智能体数量。它描述规模，不能直接代表质量；协调成本会随规模上升。
- R ：系统最终产出的结果，例如代码修改、测试报告、交接摘要或可运行产品。

1.2 软件工程的人类注意力瓶颈

演讲者指出，当前软件工程的瓶颈已经明显转向**人类注意力瓶颈**。该概念指人类在监督、审查、决策和上下文切换上的带宽限制。顶尖工程师也只能同时推进少数任务，因为每个任务都要求理解背景、判断取舍、审查提交、处理异常。即便模型可以为五十个待办功能分别生成方案，人类依旧很难同时监督五十条执行线。

可以用一个简单的吞吐关系描述这个问题。设系统实际交付吞吐为 D ，模型执行能力为 M ，人类可用于监督与验收的注意力带宽为 H ，则在缺少可靠协作架构时：

$$D \leq \min(M, H)$$

这条式子先表达一个直觉：交付速度受到较小的一侧限制。符号解释如下：

- D ：单位时间内可稳定完成并验收的工程任务量。
- M ：模型和工具链能够生成、修改、搜索、测试的执行能力。
- H ：人类用于监督实现过程、审查提交、批准范围和处理异常的注意力带宽。
- $\min(M, H)$ ：两者中的短板。若 H 很小，模型能力继续提高也难以线性转化为交付。

人类角色从盯执行转向定目标

字幕中的关键设想是：人类决定要构建什么，系统负责如何实现。这里并不意味着人类离开工程过程。更准确地说，人类注意力会更多放在目标设定、范围批准、验收标准和异常裁决上，日常执行细节则由系统持续推进。

这个瓶颈也解释了为什么“持续工作数小时甚至数天”的智能体更有意义。几十秒的智能体像一次补全；数小时的智能体才开始接近工程协作伙伴。长期运行要求系统会记录状态、会交接任务、会在失败后收束问题。如果这些能力缺失，多智能体只会把一个大任务拆成许多互相看不见的局部对话。

1.3 多智能体领域的混乱与分类法需求

当研究者和工程团队谈论多智能体框架时，很快会遇到术语混乱。不同产品使用不同角色名、不同通信模型、不同有效性判断标准。有的系统强调编排，有的强调自治；有的把多个调用步骤叫作团队，有的把一段聊天历史当作协作空间。表面上都在谈多智能体，底层结构可能差异巨大。

这种混乱有两个根源。第一，智能体的边界比传统函数或微服务更软。函数的输入输出通常明确，微服务的接口相对稳定；智能体会读取上下文、调用工具、生成计划，还会在执行中改变判断。第二，协作模式常被包装在产品叙事里。一个演示看起来像“团队协作”，实际可能只是顺序调用；另一个系统看起来像自由聊天，内部可能有严格状态机和验证规则。

先看结构，再看名称

判断多智能体架构时，最有用的三个问题是：信息从哪里来，决策由谁做，结果由谁检查。它们比“用了几个智能体”更能揭示系统本质。数量只是规模，结构才决定系统能否长期稳定工作。

因此，演讲者先提出一个简单分类法：把常见协作关系拆成五类基础模式，分别是委托、创建者-验证者对、直接通信、协商和广播。它们像电路中的基础元件：单独看都很朴素，组合之后才形成复杂系统。

1.4 五类协作模式：从分工到共享状态

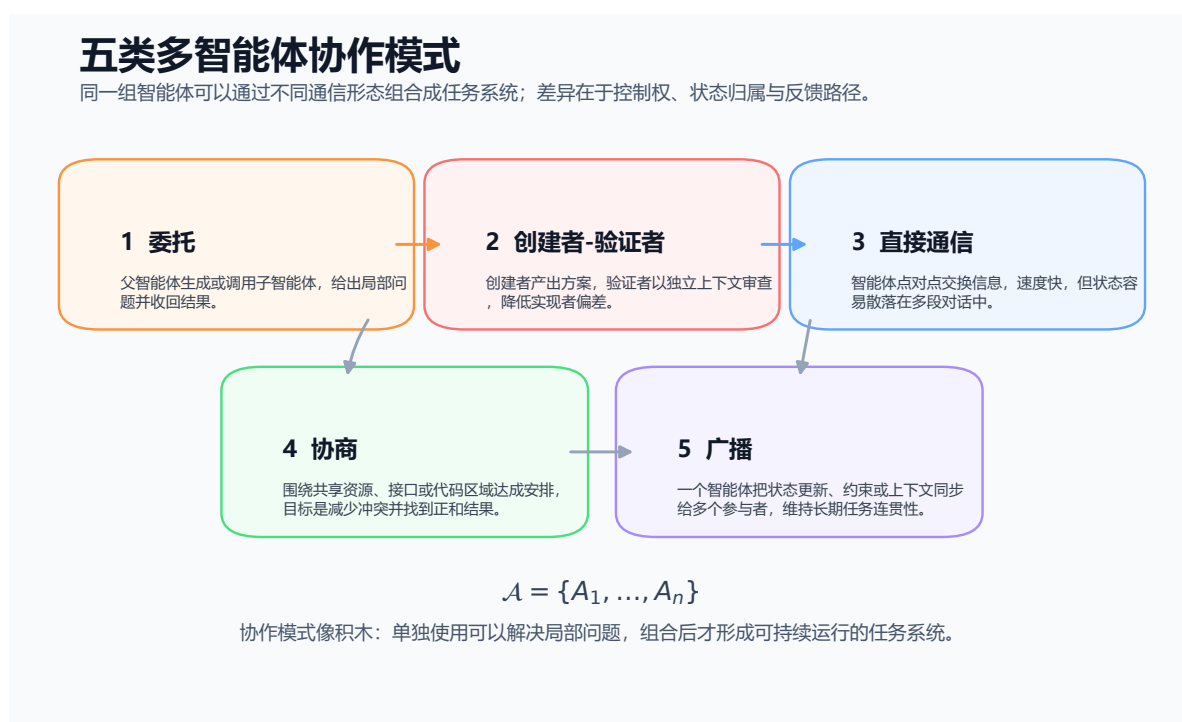


图 2：五类多智能体协作模式：委托、创建者-验证者、直接通信、协商与广播。²

委托指一个父智能体生成、调用或派出另一个子智能体来处理子任务。字幕中的例子是：父智能体要求子智能体确定数据库结构，然后把结果带回。委托是最容易实现的多智能体通信形式，因为它接近普通任务分解：上层给出局部目标，下层返回局部结果。

²概念来源时间：00:02:04-00:03:56；原幻灯片用于核对五类模式，最终图为中文矢量重绘。

委托的关键是可交付的子问题

委托的重点在于目标能被切成可独立推进的子问题，例如检索代码、调查 API、整理数据库结构、实现局部函数。若子问题边界含糊，子智能体返回的结果就很难被上层整合。

创建者-验证者对指一个智能体负责产出，另一个智能体负责检查。创建者可以写代码、生成方案或修改文件；验证者从相对独立的视角审查结果。这个模式的核心是职责分离。执行代码的智能体容易受到路径依赖影响：它知道自己为什么这样写，也更容易默认实现已经合理。验证者带着新上下文进入现场，更像代码审查者，能发现创建者忽略的问题。

验证不是补一遍测试

创建者-验证者对容易被误解为“再跑一遍测试”。测试只是验证的一种形式。更重要的是视角隔离：验证者需要检查实现是否满足目标、是否遗漏边界条件、是否引入新的风险。若验证者完全继承创建者的解释，它会继承同一组盲点。

直接通信指智能体之间进行点对点交流，中间没有中央协调者统一管理对话。它像工程团队里的私聊，局部沟通很快，适合两个角色快速交换信息。风险也很硬：状态会散落在多段对话里，长期任务很容易失真。

这里必须引入**单一事实来源**。它指系统中被所有相关角色共同承认为最新、最权威的状态记录。它可以是任务文档、计划表、数据库记录、控制台状态，也可以是结构化上下文。直接通信缺少单一事实来源时，每个智能体只知道自己那段聊天，最终就会出现“局部都合理，整体却冲突”的局面。

散落状态会放大长期任务风险

没有单一事实来源的直接通信，就像多人同时编辑一份没有版本控制的文档。每个人都以为掌握了最新信息，合并时才发现范围、接口和假设已经分叉。长期任务越长，这种分叉越难修复。

协商指智能体围绕共享资源、冲突目标或相互影响的行动进行沟通。典型场景包括多个智能体都想修改同一段代码、调用同一个 API、占用同一份预算，或者依赖同一个接口。协商并不天然指向对抗；理想情况下，它能产生正和结果，也就是参与方通过调整计划让整体收益增加。

可以用一个轻量公式表达协商的目标。假设两个智能体原本准备采取行动 x_1 与 x_2 ，系统整体收益为 $U(x_1, x_2)$ 。协商希望找到新的行动组合 x_1^*, x_2^* ，使整体收益更高：

$$U(x_1^*, x_2^*) > U(x_1, x_2)$$

符号含义如下：

- U ：系统整体收益，可综合表示交付质量、速度、稳定性和资源效率。
- x_1, x_2 ：两个智能体原本准备采取的行动。
- x_1^*, x_2^* ：协商后形成的行动组合。
- $>$ ：表示协商后的整体收益高于原先各做各的结果。

广播指一个智能体向多个智能体发送同一条信息，例如状态更新、新上下文、共享约束或范围变化。广播看起来比委托和验证低调，却对长期任务连贯性非常关键。任务运行数小时或数天时，相关角色必须同时知道边界变化；否则后续执行会沿着旧假设继续前进。

五类模式是协作积木

委托负责拆出可执行子任务，创建者-验证者对负责形成质量闭环，直接通信负责局部快速交流，协商负责处理共享资源与冲突，广播负责同步共同状态。它们构成后续任务系统的基础积木。

1.5 单一事实来源为什么决定长期连贯性

五类协作模式中，直接通信和广播共同揭示了一个容易被低估的问题：多智能体系统需要外部化的共同记忆。聊天记录可以承载局部交流，却很难天然成为全局账本。长期工程任务会不断产生新事实：范围变化、接口选择、测试失败、已完成事项、未完成风险、代码路径、命令结果。若这些事实只留在分散对话里，系统后续行动就会依赖不完整记忆。

单一事实来源的价值在于把“谁说过什么”转化为“系统现在承认什么”。前者是对话历史，后者是可执行状态。工程上，这个差别非常实际：对话历史像会议录音，里面包含大量语气、犹豫、过期判断；可执行状态像项目看板，只保留当前计划、责任人、阻塞点和验收条件。多智能体系统要能长期运行，就必须把对话沉淀成状态，把状态广播给相关角色，再让新的行动基于同一份事实。

共同记忆不是更长上下文

许多失败来自一个误判：只要上下文窗口足够长，系统就能记住一切。更长上下文能缓解遗忘，却不能自动解决事实冲突、状态过期和责任边界问题。共同记忆需要结构化记录、更新规则和权威来源；它更像数据库或项目账本，功能不同于一段无限延长的聊天记录。

这正是第 1 章通向第 2 章的桥。到这里，演讲者只是建立了“积木层”的词典。更难的问题是：这些积木怎样拼成一个能连续运行多日的工程系统。下一章将进入 Factory 的任务系统，说明目标澄清、计划批准、角色分工和长期执行如何被组织成更稳定的工作流。

1.6 本章小结

- 演讲者从 Goose 和 Factory 的经验出发，把多智能体架构放进软件开发生命周期自主化的语境中讨论。
- 本章的核心判断是人类注意力瓶颈：模型执行能力增加后，监督、审查、决策和交接会成为新的短板。
- 智能体是具备目标、上下文、工具调用和局部自主决策能力的软件执行单元；多智能体系统是多个智能体通过通信、分工、验证和共享状态共同完成任务的系统；智能体团队进一步强调角色互补、责任边界和协作协议。
- 多智能体领域容易混乱，因为不同框架使用不同术语、角色边界和状态管理方式；判断系统时应优先观察信息来源、决策主体和检查机制。
- 五类基础协作模式分别是委托、创建者-验证者对、直接通信、协商和广播；它们构成后续工程系统的协作积木。
- 单一事实来源是长期任务的稳定器。缺少共同状态记录时，直接通信会让事实散落在对话中，后续执行容易沿着过期假设继续推进。

2 Factory 任务系统与三角色架构

第 1 章已经把多智能体协作拆成若干基础积木：委托、创建者-验证者对、直接通信、协商与广播。进入第 2 章后，演讲者把问题推进到工程系统层面：这些积木怎样组合成一个能够连续运行数小时甚至数天的任务系统。真实软件开发很少停留在一次问答里；它通常包含目标描述、范围澄清、计划批准、功能实现、阶段验收和后续交接。只要其中一个环节缺少稳定边界，长期任务就会在执行中逐渐失去方向。

2.1 从协作积木到任务系统

Factory 在本章指演讲者所在公司及其任务系统语境，核心目标是让软件开发生命周期中的更多环节可以被智能体持续推进。这里的 **Factory** 不应被理解为传统流水线工厂；更贴切的类比是一个工程调度室：人类提出目标，系统把目标转化为计划、角色分工和阶段检查点，再让不同智能体按约束推进。

任务系统，也就是字幕中出现的 **Mission system**，是把目标、范围澄清、计划、执行、验证和状态交接组织到同一工作流中的多智能体工程机制。本文后续统一写作“任务系统 / Mission”：Mission 强调一段有目标、有边界、有完成条件的长期工程行动，任务系统强调支撑这段行动的流程、角色和记录机制。它的入口可以是一段聊天，但它的主体是一套可持续运转的工程生态。

演讲者给出的工作流很直接：用户先描述目标，系统通过对话明确范围，随后生成计划并请求批准；计划获批后，系统可以持续运行数小时或数天。这个过程保留了**人在环路中**：人类负责目标设定、范围判断、计划批准和关键干预，把注意力放在方向性决策上，而非持续盯着每一步执行。换言之，人类仍然是任务的责任主体，只是监督方式从“贴身盯执行”变成“在关键边界上批准、检查和纠偏”。

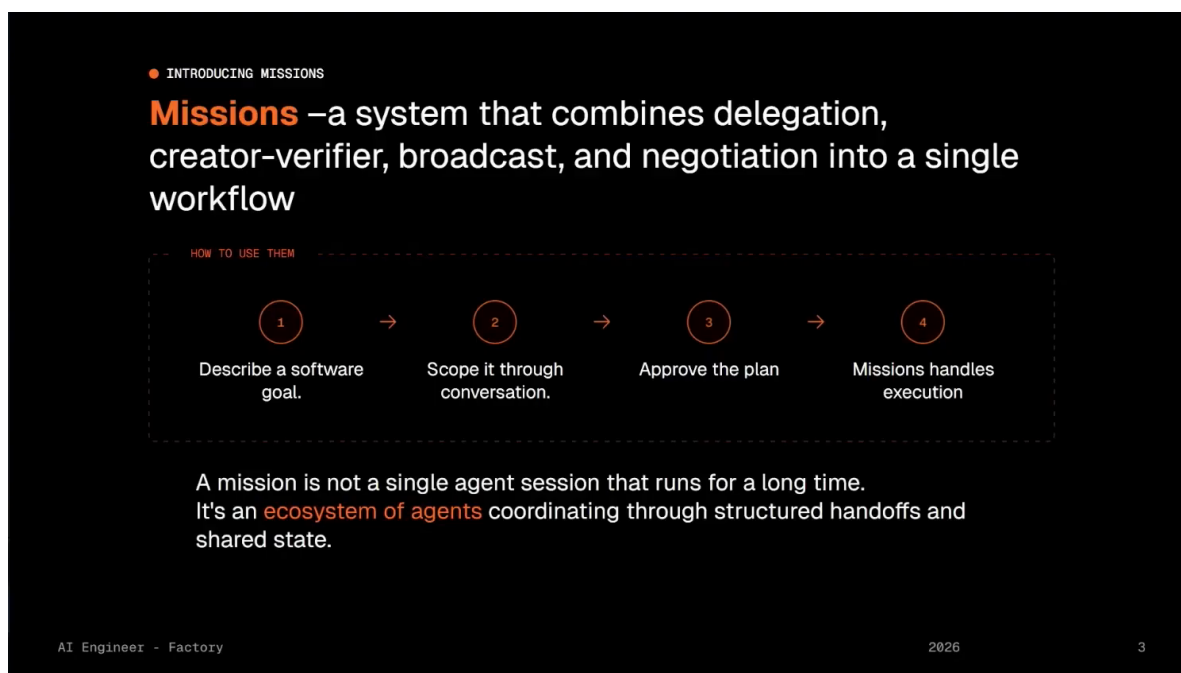


图 3: 任务系统工作流：描述目标、对话澄清范围、批准计划，再由系统持续执行。³

³视频画面时间来源：00:04:03–00:04:35；最终采用帧 00:04:31。

任务系统的核心定义

任务系统 / Mission 是一个把自然语言目标转化为可执行工程过程的机制。它先澄清目标和范围，再形成计划，随后沿阶段边界推进实现与检查。它的价值在于让状态、职责和验收边界保持稳定，使多智能体系统能够从短对话扩展到长时间工程执行。

任务分解是任务系统的第一项关键动作：它把一个较大的目标拆成计划、里程碑、功能项与验证条件。这个动作像把一张模糊地图改写成施工图。地图只告诉人“要到哪里”，施工图还要说明道路、材料、顺序、检查点和返工条件。若缺少任务分解，系统看似很勤奋地生成代码，实际可能只是在不确定的方向上加速。

计划是经过范围澄清后的执行路线，后文记为 P 。它说明哪些功能要做、按什么顺序推进、每个阶段怎样判断完成。**里程碑**是长期任务中的阶段边界，后文用 M_k 表示第 k 个里程碑。里程碑比普通待办项更强，它同时承担三件事：确认阶段成果、触发验证、为后续角色提供交接边界。

在符号上，可以把第 1 章已经出现的目标 T 交给协调器处理，并生成计划和里程碑序列。这里先给出直观公式，随后解释每个符号：

$$C_o(T) \longrightarrow P = \{M_1, M_2, \dots, M_n\}.$$

其中， C_o 表示协调器，使用下标 o 是为了避免与后文“上下文”的符号混淆； T 表示人类给出的任务或目标； P 表示计划； M_k 表示第 k 个里程碑。这个公式承担系统图例的作用：协调器把目标加工成计划，计划由一串可检查、可交接的阶段边界组成。

2.2 三角角色架构

任务系统采用三角角色架构：**协调器**、**工作者**、**验证者**。这三个角色分别对应规划、执行和检查。它们共同解决长期任务中的三个核心风险：目标起点含糊、实现过程漂移、结果只在局部正确。

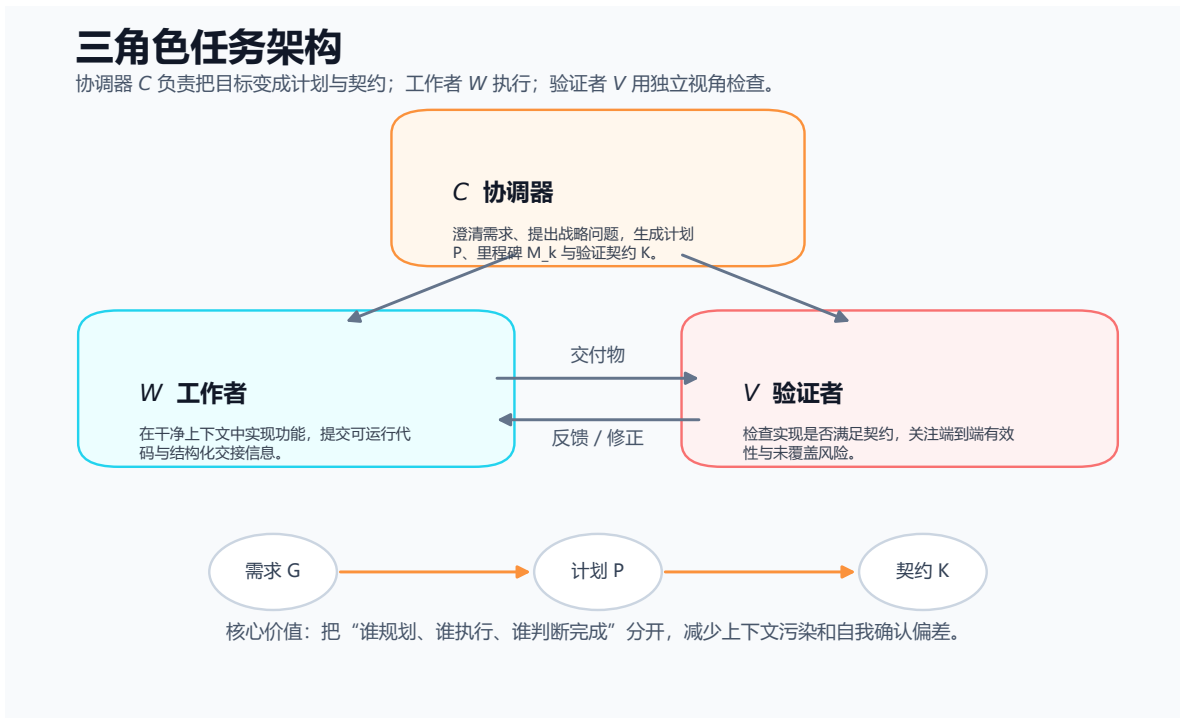


图 4：三角角色架构：协调器 C_o 生成计划与契约，工作者 W 执行，验证者 V 独立检查。⁴

协调器记为 C_o ，负责澄清需求、提出战略问题、生成计划 P 、划分里程碑 M_k ，并在计划中写入后续验证所需的成功条件。字幕中把协调器类比为“思想顾问”：当用户描述需求时，它不会急着把第一句话翻译成代码任务，而会检查问题空间中的模糊点。例如用户说“做一个 Slack 克隆”，协调器需要继续追问实时消息、频道权限、文件上传、部署范围和验收路径。每一个答案都会改变计划 P 。

协调器 C_o 的职责

协调器 C_o 接收目标 T ，通过提问降低歧义，输出计划 P 和里程碑 M_k 。它的重点职责包括澄清范围、暴露隐含假设、安排执行顺序，以及为后续验证留下清晰依据。

工作者记为 W ，负责在干净上下文中实现功能并提交结果。工作者也可理解为执行者，但本章统一使用“工作者”，强调它承担具体工程劳动：阅读规格、修改代码、运行必要检查、提交可交接成果。演讲者特别强调，工作者拿到的是清晰上下文，没有累积负担，也没有注意力下降。对长期任务而言，这一点很硬核：旧决策、旧错误和半完成想法会污染后续执行；让 W 在整理后的上下文中工作，相当于给每个阶段重新铺好桌面。

干净上下文为何重要

干净上下文并不意味着信息贫乏。它更像一张整理好的工单：包含当前功能需要的规格、约束和最新代码状态，同时去掉无关历史。对智能体而言，这还意味着 token 预算不会被旧讨论塞满，注意力可以集中在当前里程碑 M_k 上。

验证者记为 V ，负责检查工作者输出是否满足目标。普通检查会运行测试、类型检查或代码审查；Factory 任务系统更进一步，要求验证者关注**端到端有效性**。端到端有效性指结果是否从用户目标出发，穿过计划、实现和验证之后，真的完成了用户可感知的工作。它像餐厅出餐：切菜、炒制、摆盘每个局部动作都合格，顾客拿到的整道菜仍然可能因为送错桌或缺主菜而失败。软件也是如此，局部测试通过不能自动推出用户流程可用。

验证者 V 的职责

验证者 V 把检查目标从局部代码正确性提升到端到端有效性。它需要判断 W 的成果是否对齐目标 T 、是否符合计划 P 的阶段要求、是否让当前里程碑 M_k 可以进入下一步。

三角架构可以用下面的轻量公式概括。公式前半段表示规划，后半段表示执行与检查之间的闭环：

$$C_o(T) \rightarrow (P, M_k), \quad W(P, M_k) \rightarrow R_k, \quad V(R_k, T, P, M_k) \rightarrow E_k.$$

这里， R_k 表示工作者在第 k 个里程碑提交的阶段结果， E_k 表示该阶段的端到端有效性判断。符号关系说明： C_o 负责把目标变成计划和里程碑； W 按计划实现当前里程碑； V 拿阶段结果与目标、计划、里程碑对照，判断它是否真的有效。三者之间的分工让任务系统不会只依赖某一个智能体的自我判断。

2.3 结构化交接与生态视角

演讲者特别提醒，Mission 超出了单次会话的范围，它是通过结构化交接共享状态的智能体生态系统。本章只点明这个生态视角，不展开第 3 章的交接机制细节。关键在于：长期任务需要把状态放在

⁴概念来源时间：00:04:36-00:05:12；原幻灯片用于核对三角色关系，最终图为中文矢量重绘。

可传递、可检查的位置。单次聊天像临时会议，适合快速沟通；任务系统更像项目档案，适合跨阶段、跨角色保存责任边界。

这一点解释了为什么协调器、工作者、验证者需要围绕里程碑协作。协调器定义 M_k ，工作者围绕 M_k 提交成果，验证者在 M_k 边界检查端到端有效性。若验证通过，系统进入下一个里程碑；若验证失败，系统获得明确的返工位置。这样，长期运行不会变成一串散落的聊天记录，而会成为一条可追踪的工程链路。

常见误解：把任务系统看成更长的聊天窗口

更长的聊天窗口只能容纳更多文本，不能天然提供责任分离、阶段边界和验收判断。任务系统真正要解决的是“谁来规划、谁来执行、谁来验证、状态在哪里被交接”。这些问题若没有结构化答案，窗口再长也会堆积混乱。

2.4 本章小结

本章说明了 Factory 任务系统怎样把第 1 章的协作积木组合成长期工程流程。任务系统 / Mission 从目标描述开始，经由范围澄清和计划批准，进入持续执行；人在环路中负责目标、范围、批准和关键干预。

三角色架构给出了稳定分工：协调器 C 。负责澄清需求并生成计划 P 与里程碑 M_k ；工作者 W 在干净上下文中实现功能；验证者 V 检查阶段结果是否具备端到端有效性。下一章将继续追问：验证依据怎样在编码前被定义，以及长期任务怎样通过交接机制减少状态丢失和实现偏差。

3 验证合约、对抗性验证与结构化交接

上一章已经交代任务系统中的三类角色：协调器负责把目标澄清为计划和里程碑，工作者负责实现，验证者负责检查端到端有效性。本章继续追问一个更硬的问题：长期运行的智能体系统怎样判断“做对了”，又怎样避免执行过程中的上下文逐步偏离目标。演讲者给出的答案有三层：预先写下验证合约，用独立验证器进行对抗性检查，再通过结构化交接保存任务记忆。

这里先定义上下文。上下文指智能体在执行时可见的目标、历史、代码状态、约束、命令结果、错误信息和交接摘要。它像工程现场的白板、图纸、施工日志和验收单的总和。上下文越长，信息越容易被稀释；上下文越乱，智能体越容易把局部实现当成真实目标。于是，验证和交接属于长期任务保持方向感的基础设施。

3.1 实现后补测试的盲区

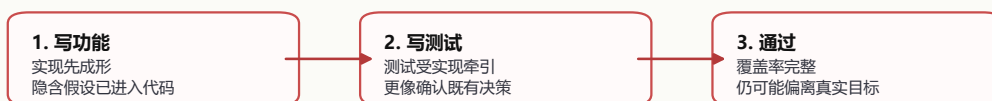
许多编码智能体会呈现一种看起来很顺滑的流程：先构建功能，再补测试；测试通过，覆盖率也完整。演讲者指出，这种流程的危险在于测试经常受实现影响。代码先给出答案，测试再围绕这个答案组织语言，最后验证环节只是在确认实现者已经做出的决策。

这种盲区可以用“验收单”和“施工记录”的差别来理解。验收单应当在施工前写清楚：门要多宽，电路要承载多少功率，消防通道要满足什么规范。施工记录发生在施工后，主要说明施工队已经怎样做。若系统先实现再补测试，测试很容易从施工记录里长出来，难以独立代表原始目标。

实现后补测试 vs 预先验证契约

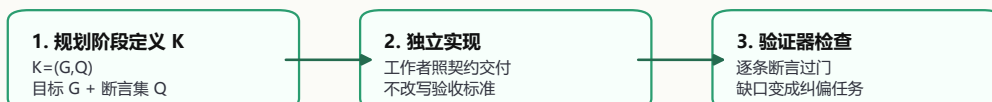
核心差异：正确性来源于代码，还是被实现结果反向塑形。

后补测试路径



风险：测试像回声室，声音很响，却来自同一个源头。

验证契约路径



收益：契约像验收单，先写清交付标准，再检查交付物。

图 5: 实现后补测试与预先验证合约的流程差异。后补测试容易被既有实现牵引；验证合约在编码前写清任务或目标 T 与断言集 Q ，让实现与验证共同围绕 $K = (T, Q)$ 收敛。⁵

实现后补测试的确认偏差

确认偏差在这里指验证材料受到已有实现影响，只检查实现者已经想到的路径。覆盖率高只能说明代码路径被执行过，不能自动说明用户目标被满足。测试覆盖率像地图上被荧光笔涂过的路，真正的问题是这些路是否通向目的地。

从工程角度看，这个盲区会缓慢积累。一次功能偏一点，下一次测试又确认这个偏差，长期任务就会沿着错误方向稳定前进。稳定性与正确性是两种性质：系统可以很稳定地走错路。多日运行的任务尤其需要在实现前确定验收标准，因为后补测试越多，越可能把错误路线装修得很像正路。

3.2 验证合约

验证合约指协调器在编码开始前定义的一组成功条件和可检查断言，用来说明目标怎样才算完成。这里使用“合约”一词，是为了强调预先约定、可检查、后续执行不能随意改写。它像工程项目中的验收单，也像 API 设计里的接口约定：工作者可以自由选择内部做法，但交付物必须满足外部约定。

本章按全局符号表把验证合约写成：

$$K = (T, Q), \quad Q = \{q_1, \dots, q_m\}.$$

其中， K 表示**验证合约**； T 表示一个任务或目标，也就是人类或协调器希望系统完成的工作； Q 表示**断言集合**，也可理解为质量门集合； q_j 表示第 j 条断言，即一条可检查的正确性条件或质量条件； m 表示断言数量，复杂项目中 m 可以达到数百。公式里的 $Q = \{q_1, \dots, q_m\}$ 说明合约应拆成一组可以逐条核验的检查点，避免停留在笼统愿望层面。

断言描述“必须成立的事实”。它和测试用例相关，但二者不等价。比如“普通用户不能访问管理员页面”是一条断言；用浏览器登录普通账号、输入管理员 URL、期待 403 或重定向，是一条测试路

⁵ 视频概念来源：00:06:06–00:06:55。

径。一条断言可以对应多条测试路径，一条测试路径也可能覆盖多条断言。

验证合约先于实现定义正确性

验证合约的关键价值在于独立性：正确性先由目标 T 和断言集合 Q 定义，再由工作者实现。验证者检查的是“实现是否满足合约”，测试不会反过来迎合实现。可以把 $K = (T, Q)$ 看成验收单： T 写明要交付什么， q_j 逐条写明验收员要检查什么。

演讲者强调，复杂项目可能包含数百条断言，并且每个功能都会分配一个或多个必须满足的断言。所有功能组合起来，必须覆盖整个断言集合 Q 。换成更朴素的话，系统不能只把容易验证的部分做漂亮，还要保证每条质量门都有责任归属。

断言覆盖与代码覆盖

断言覆盖关注哪些正确性条件被负责过；**代码覆盖**关注哪些代码路径被运行过。前者偏向目标外部，后者偏向实现内部。多智能体系统需要二者，但优先级不同：缺少断言覆盖时，代码覆盖再高也可能只是把错误实现跑了一遍。

本章还需要区分**评估**与**反馈**。评估是检查交付结果是否满足验证合约，例如某条 q_j 是否通过；反馈是把失败信息、证据和修正方向送回系统，让后续工作者能够纠偏。评估像体检报告，反馈像治疗方案和复查要求。只有评估没有反馈，系统只能知道失败；有了反馈，系统才有机会修复失败。

3.3 两类验证器

在每个功能里程碑之后，系统会运行两类验证器：**审查验证器**与**用户测试验证器**。二者都服务于同一个验证合约 K ，但检查角度不同。

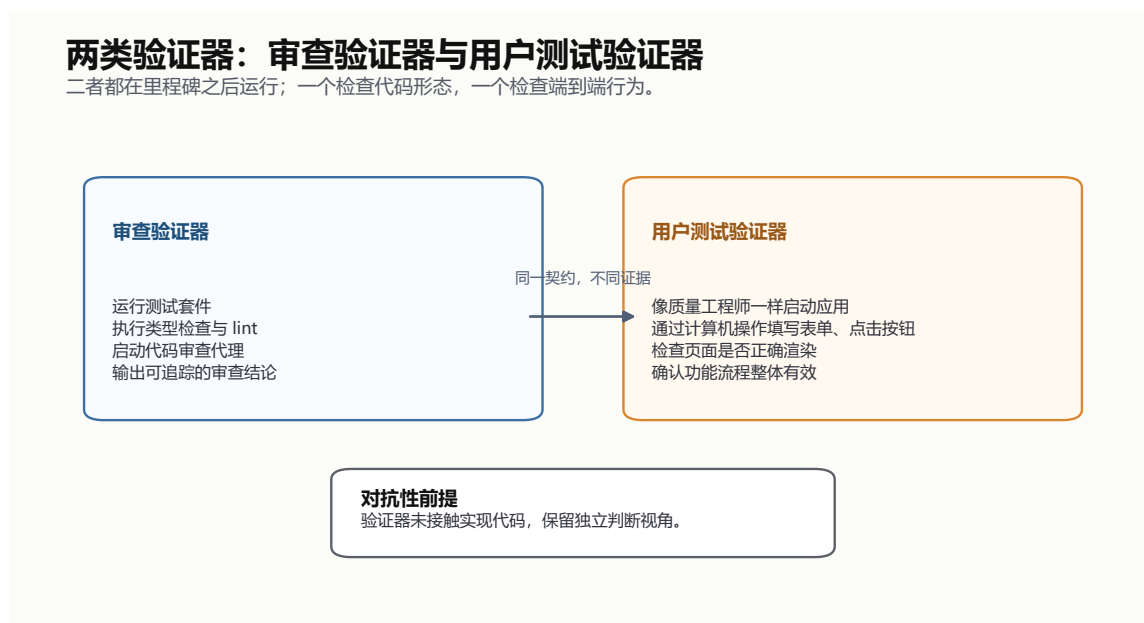


图 6: 两类验证器的职责分工。审查验证器负责测试、类型检查、lint 与代码审查；用户测试验证器像质量工程师一样启动应用、操作界面并检查端到端流程。⁶

⁶ 视频概念来源：00:06:57–00:08:08。

审查验证器更接近传统软件工程里的质量门。它会运行测试套件、类型检查，进行严格代码审查，并且可以为每个完成的功能启动专用代码审查智能体。它关心的问题包括：测试是否失败，类型是否一致，接口是否被破坏，错误处理是否缺失，代码是否明显违背已有架构。

用户测试验证器更像质量工程师。它会启动应用程序，通过计算机操作与实时应用交互：填写表单、检查页面是否正确渲染、点击按钮、走完整个功能流程。它检查的对象不只包括代码仓库里的文本，还包括界面状态、交互反馈、端到端流程和用户可感知结果。

两类验证器覆盖两种失败面

审查验证器主要覆盖代码侧失败面：类型错误、测试失败、接口破坏、明显设计缺陷。用户测试验证器主要覆盖体验侧失败面：页面未渲染、按钮无响应、流程走不通、真实应用状态不符合预期。前者像实验室检测，后者像现场试车。

用户测试验证器通常更耗时，因为它要等待真实应用运行，而非只消耗模型生成 token。这里有一个容易被低估的工程事实：真实软件的时间成本经常花在启动服务、等待构建、浏览器交互、数据库迁移和外部依赖响应上。多智能体系统如果只优化模型说话速度，就会误判整条交付链路的瓶颈。

token 时间与墙钟时间

token 时间指模型生成和读取文本消耗的预算；**墙钟时间**指现实世界经过的时间。用户测试验证器消耗更多墙钟时间，因为它必须等待应用真正启动、渲染和响应。工程交付看的是墙钟时间，用户也只感受到墙钟时间。

若用符号表示验证结果，可以把验证者 V 对合约 K 的通过记为 $\text{pass}(V, K)$ 。这只是概念记号，并非数学证明：验证者根据合约检查交付物，确认相关断言已经满足。生产系统更关心失败时的反馈：哪条 q_j 失败，失败证据是什么，修正范围在哪里。

3.4 对抗性验证

对抗性验证指验证者保持独立视角和独立上下文来检查工作产物。这里的“对抗性”并不指安全攻防，也不指组织内部互相拆台；它指一种制度化的不轻信：验证者不被实现者的推理路径、临时借口和局部假设带偏，必须从合约和用户结果出发重新审视交付物。

演讲者提到一个关键设计：验证器没有接触过代码生成过程。换句话说，验证者拿到的是交付物、合约和必要环境，而非工作者脑中的解释。这个隔离让验证者更容易发现实现者看漏的问题。

独立上下文需要保留必要事实

对抗性验证需要隔离实现偏见，但不能切断必要事实。验证者仍然需要读取验证合约 K 、相关断言 q_j 、运行方式和期望行为。真正要隔离的是工作者的自我辩护和未验证假设，而非验收所需的证据。

本章用 C 表示上下文。工作者的 C 通常包含实现过程、局部取舍和临时解释；验证者的 C 应围绕合约、运行环境和可观察结果组织。二者差异构成对抗性验证的基础。

还需要定义**上下文漂移**。上下文漂移指任务在长时间执行中逐步偏离原始目标、约束或事实状态：早期假设被忘记，失败命令被忽略，未完成事项被误认为已完成，局部实现叙事盖过了真实用户目标。它像航行中的微小偏航，短距离看不明显，长距离会把船带到完全不同的位置。

对抗性验证切断实现者偏见

对抗性验证让验证者从 K 和可观察行为出发，而非从工作者的实现叙事出发。它把“工作者认为完成了”转换成“独立验证者确认断言成立”。这个转换是多智能体系统走向可靠交付的关键。

3.5 结构化交接

验证能够发现错误，但多日任务还面临另一个更沉默的风险：上下文丢失。一个任务若持续十六天，智能体不可能只依赖当前对话窗口记住所有决策、命令、失败尝试和未完成事项。系统因此需要**结构化交接**。

结构化交接是在工作者完成功能或到达里程碑边界时填写的标准化文档机制，用于跨里程碑传递上下文。它记录已完成内容、未完成事项、执行过的命令、这些命令的**退出码**、发现的问题，以及是否遵守了协调器定义的流程。退出码是程序结束时返回给操作系统的状态编号，常见约定是 0 表示成功，非零值表示出现错误或异常。它像机器给出的简短收据：命令有没有按预期结束，先看退出码，再看日志细节。

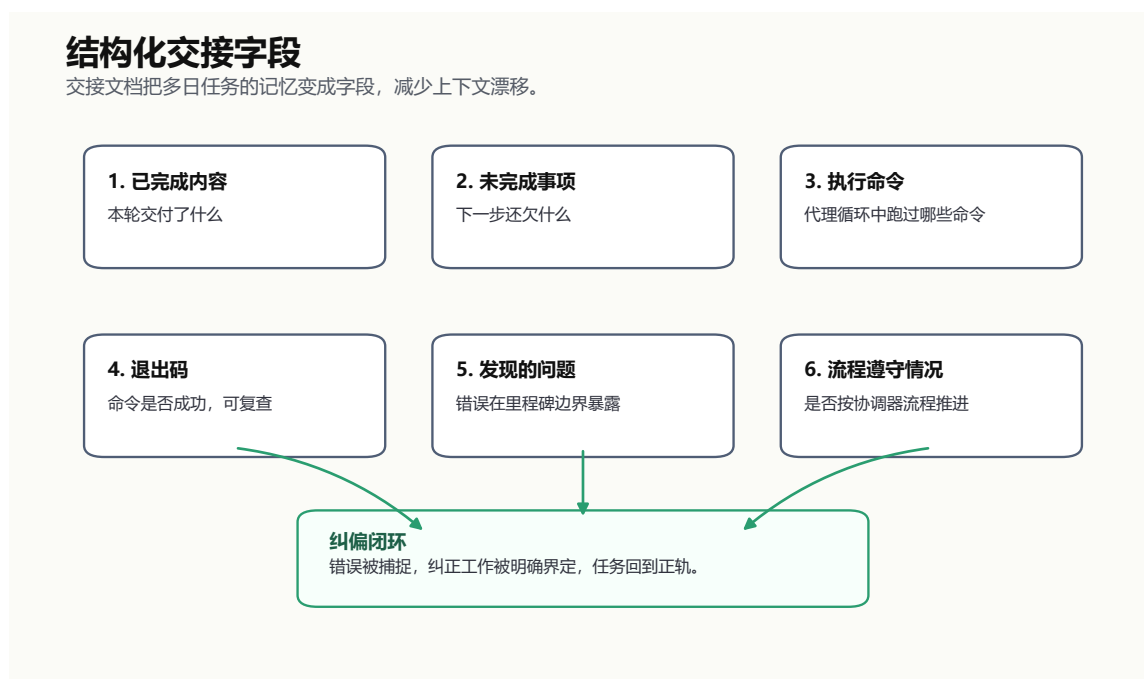


图 7: 结构化交接文档的关键字段。该文档把完成内容、未完成事项、执行命令、退出码、发现的问题与流程遵守情况固定下来，让多日任务在里程碑边界保留可复查的上下文。⁷

本章用 $\mathcal{H}$ 表示结构化交接文档。一个实用的 $\mathcal{H}$ 至少应包含以下字段：

- 已完成内容：本轮实际交付了什么，涉及哪些功能或文件。
- 未完成事项：哪些目标仍未完成，哪些断言还没有验证。
- 执行命令：工作者在智能体循环中运行过哪些命令。
- 退出码：每条关键命令的状态编号，以及失败时的错误摘要。
- 发现问题：实现、测试、环境、需求理解中的异常和风险。

⁷ 视频概念来源：00:08:12–00:09:16。

- 流程遵守情况：是否按协调器的计划、里程碑和验证合约执行。

结构化交接是外部记忆

$\mathcal{H}$ 可以看成任务系统的外部记忆。自然语言对话像人的短期记忆，容易被新消息挤出；结构化交接像项目日志，把关键事实固定成可读取、可审计、可继续执行的记录。长期任务能否跨天运行，经常取决于外部记忆是否可靠。

交接文档的价值不止于“留痕”。它让系统具备**自我修复能力**。自我修复指系统在里程碑边界捕捉错误，根据验证失败、交接记录和反馈明确修正范围，再把任务拉回目标轨道。用符号表示，可以把偏差或修正方向记作 Δ ：当某条断言 q_j 失败，系统根据 $\mathcal{H}$ 和验证结果确定 Δ ，再让后续工作者执行这部分修正。

这里要注意一个细节：自我修复发生在里程碑边界，效果最好。边界像航海中的定位点，船走一段就测一次位置；如果偏了，就在偏差还小的时候修正。缺少边界时，错误会混入后续工作，最后很难分清哪一步开始偏航。

长期任务依赖验证边界与交接记忆

里程碑提供边界，验证合约提供验收标准，结构化交接提供任务记忆。三者合在一起，系统才能在发现错误时明确修正范围，让任务回到原目标轨道。

演讲者提到，Factory 最长的任务持续了十六天，并且他们相信可以运行三十天。这个数字的重点不在炫耀运行时长，而在说明架构性质：只要验证和交接足够稳定，任务可以跨越单次上下文窗口、单个智能体轮次，甚至跨越普通软件团队的完整冲刺周期。

无结构总结会制造新的噪声

“写一段总结”通常不够。自由文本容易漏掉命令、退出码、失败证据和未完成断言。结构化交接要求固定字段，原因很直接：后续智能体需要按字段恢复状态，不能在大段叙述里猜重点。

3.6 本章小结

- 实现后补测试容易受已有代码影响，测试通过和覆盖率完整都不能自动证明目标达成。
- 验证合约 $K = (T, Q)$ 把任务或目标 T 与断言集合 $Q = \{q_1, \dots, q_m\}$ 写在实现之前，其中 q_j 表示第 j 条可检查质量条件。
- 评估负责判断交付结果是否满足合约，反馈负责把失败证据和修正方向送回系统。
- 审查验证器负责代码侧质量门，用户测试验证器负责真实应用侧流程，二者共同检查合约是否成立。
- 对抗性验证通过隔离验证者上下文 C ，减少实现者偏见和上下文漂移，让验证从合约和可观察行为出发。
- 结构化交接文档 $\mathcal{H}$ 保存完成内容、未完成事项、命令、退出码和发现的问题，使多日任务保留外部记忆。
- 自我修复依赖清晰边界：当某条断言失败，系统需要根据验证结果与交接记录明确修正方向 Δ ，再把任务拉回原目标。

4 运行策略、目标内并行与任务控制台

上一章已经说明，长期任务能持续运行，依赖验证合约、对抗性验证和结构化交接。它们解决的是“怎样判断做对了”和“怎样把上下文传下去”。本章接着处理一个更硬的工程问题：当系统里有多个智能体时，任务到底应该怎样调度，才能减少冲突、重复劳动和预算浪费。

演讲者在这一段给出的结论很克制。多智能体系统的价值并不等于把所有智能体同时放出去改代码。软件开发的共享状态太密集，代码库、接口约定、数据模型和产品语义彼此咬合；如果所有人同时动手，表面上的吞吐量很快会被冲突和返工吃掉。因此，本章的核心是运行策略：功能主线保持串行，局部研究、搜索和验证可以并行；人类通过任务控制台观察状态、预算和交接摘要，以项目经理或批准者身份低频介入。

4.1 全局并行的诱惑

当任务系统已经拥有协调器、工作者和验证者之后，一个直觉做法是全局并行：十个智能体同时运行，理论上似乎可以得到十倍吞吐。这种想法很诱人，因为它把软件工程想象成简单的搬砖队列：只要增加工人，墙就会更快砌好。

软件开发的现实更接近多人同时改同一张城市管网图。一个智能体调整认证流程，另一个智能体修改路由结构，第三个智能体迁移数据库 schema；它们看起来分工明确，实际共享同一套代码、同一组设计假设和同一条产品语义。只要共享状态发生变化，后续工作就必须重新对齐。对齐动作本身会消耗时间、token 和人类注意力。

全局并行容易把吞吐幻觉变成返工账单

全局并行最危险的地方，是把“同时运行”误读成“同时有效推进”。多个智能体同时写同一套系统状态时，冲突修改、重复劳动和架构分叉会快速累积。吞吐曲线可能短暂上升，随后被合并成本、验证失败和返工成本压下来。

这一点和数据库并发控制很像。读取数据通常可以扩散，写入共享状态必须有事务边界、一致性协议和冲突处理。软件代码库也是共享状态，只是它的冲突不都表现为 Git 合并失败；更麻烦的冲突常常是语义冲突：两个修改都能编译，组合起来却破坏了产品行为。

4.2 协调成本如何被并行放大

本章首次定义**协调成本**：它指多个智能体为了避免互相冲突、保持设计一致、同步上下文和修复并行副作用而额外付出的系统代价。这个代价包含沟通时间，也包含 token 消耗、重复搜索、冲突合并、验证返工、架构决策不一致和人类重新审查的负担。

可以用 workflow 符号把问题说清楚。令 W_f 表示一个 workflow， B 表示预算；这里的预算可以是 token、墙钟时间、计算资源或人类审查带宽。对一个长期软件任务，预算消耗可以粗略写成：

$$B(W_f) \approx B_{\text{work}} + B_{\text{coord}} + B_{\text{rework}}$$

其中， $B(W_f)$ 表示 workflow W_f 的总预算消耗； B_{work} 表示真正用于实现、研究和验证的预算； B_{coord} 表示同步状态、统一设计方向和解决冲突的协调预算； B_{rework} 表示并行冲突或错误假设导致的返工预算。

公式的含义很直接：全局并行主要压低的是单个任务的等待时间，却会抬高 B_{coord} 和 B_{rework} 。如果十个智能体同时写代码，它们之间潜在的关系接近 $\frac{10 \times 9}{2} = 45$ 对；每一对都可能产生接口、文件、

语义或计划层面的互相影响。并行度从 n 增加时，协调关系常按近似 $O(n^2)$ 增长，而可交付功能未必按 $O(n)$ 增长。

预算不是只看模型账单

本章的 B 不只表示 token 费用。工程预算还包括等待真实应用启动的墙钟时间、验证失败后的返工时间、人类重新理解上下文的注意力成本，以及冲突合并造成的机会成本。只盯 token 单价，会低估长期任务中最贵的部分。

字幕里的描述相当明确：他们尝试过这种并行方式，在软件开发任务中效果不好，因为智能体会互相覆盖修改、重复劳动，并做出不一致的架构决策。更糟的是，这些问题经常到验证阶段才暴露；此时系统已经花掉大量预算，错误假设也可能写进多个模块。

全局并行与目标内并行

并行的关键约束：写入路径要少而清楚，只读工作可以局部铺开。

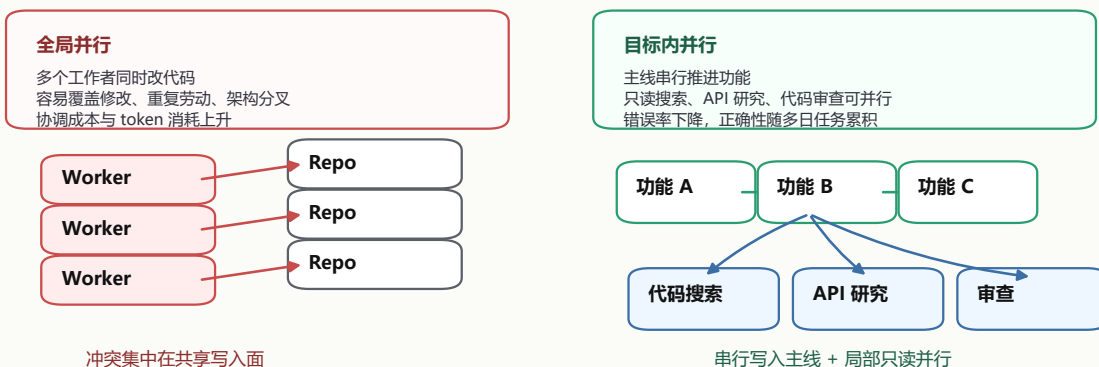


图 8: 全局并行与目标内并行的运行策略对比。全局并行会放大写入冲突、重复劳动和架构分叉；目标内并行保留串行写入主线，把代码搜索、API 研究、代码审查等只读工作局部并行化。⁸

4.3 串行执行与目标内并行

本章首次定义**串行执行**：它指功能主线以单线程方式推进，在任一时刻只让一个工作者，或一个被明确界定的有效错误修复流程，持有主要写入权。这里的“串行”并不表示系统只能做一件事；它强调共享代码状态的主干写入要有顺序，像单车道桥梁需要排队通过，避免两辆车在桥中央互相卡住。

与串行执行配套的是**目标内并行**。目标内并行指围绕同一个目标或里程碑，在不破坏共享状态的边界内并行执行局部工作，例如代码库搜索、API 研究、文档查阅、依赖调查、代码审查和验证分析。它服务当前目标，帮助主线更快获得证据，却不把整个项目拆给一群智能体同时写。

进一步说，**只读并行**是目标内并行中最安全的一类：多个智能体可以同时读取代码、搜索资料、检查日志、分析测试失败或审查变更，但不直接写入代码库。只读并行像勘探队，可以同时测绘地形、标记风险、收集证据；写入主线像施工队，需要明确顺序和责任边界。

⁸ 视频概念来源：00:09:17–00:10:28。

读写边界是并行策略的分水岭

读取操作通常可以复制上下文、分散搜索、互不干扰；写入操作会改变共享状态，一旦多个写入者缺少边界，就会引入合并冲突、语义冲突和架构漂移。多智能体软件工程用串行写入主线、里程碑检查和结构化交接来降低这种风险。

演讲者把这种策略称为带目标内并行的串行执行。纸面上它显得慢，因为同时改代码的智能体变少了；长期任务中它往往更稳，因为错误率下降，返工减少，交接更清晰。正确性会累积，错误也会累积。一个运行数天的任务，真正要优化的是每个里程碑之后系统是否更接近可交付状态，单看某一分钟里有多少智能体在忙会误导判断。

目标内并行的判断标准

目标内并行的核心判断是：并行工作能否保持共享状态稳定。代码搜索、API 研究、文档查阅、审查验证适合并行；对同一代码库主干的写入应当严格排队。换成交通类比，侦察车可以多路出发，施工车辆需要统一调度。

4.4 任务控制台

当任务持续数小时甚至数天，标准聊天界面会变得笨重。聊天记录擅长承载问答，却很难回答项目管理问题：当前进度到哪个里程碑，预算用了多少，哪个智能体正在工作，最近一次交接说明了什么，验证者发现了哪些问题，任务是否已经偏离最初目标。

本章首次定义**任务控制台**：它是面向长期智能体任务的专用状态视图，用来展示任务进度、预算消耗、当前活跃智能体、交接摘要、验证状态和后续路径调整。它的作用类似机场塔台或 CI/CD 仪表盘。塔台不亲自驾驶每架飞机，却必须知道哪架正在起飞、哪架等待、跑道是否被占用；任务控制台也不替工作者写代码，却让人类快速恢复对系统状态的把握。

任务控制台把对话流变成状态面板

长期任务不能只藏在聊天记录里。任务控制台把活跃智能体、预算、交接摘要、验证结果和路径调整抽出来，变成可扫描、可追踪、可审计的状态面板。这样人类不用翻完整对话，也能判断任务是否仍在正确轨道上。

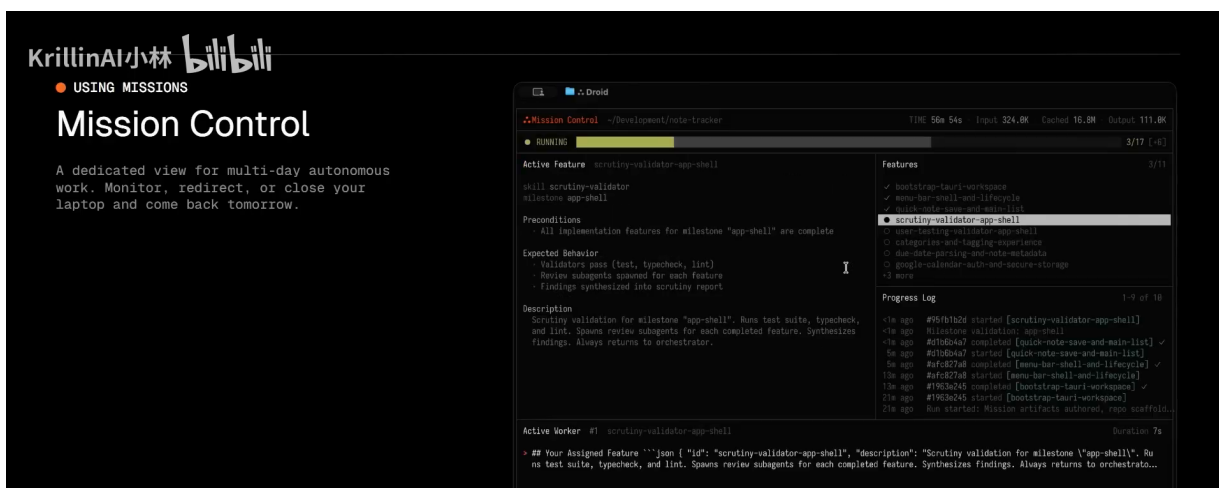


图 9: 任务控制台的视频帧。画面显示 Mission Control 中的进度条、Active Feature、Features 列表、Progress Log 与 Active Worker 区域，说明多日任务需要专门视图来观察状态、预算、交接摘要和活跃智能体。⁹

任务控制台中的**预算**，对应符号 B 。它既可以表示 token 预算，也可以表示时间预算、计算预算和人类审查预算。控制台关心的是任务是否在预算 B 内朝目标推进，而单条回复看起来是否流畅只具有局部意义。如果预算已经明显超支，人类需要介入重新收窄范围、调整里程碑，或要求协调器重新规划。

交接摘要在控制台中尤其关键。上一章的结构化交接记录完成内容、未完成事项、命令、退出码和发现的问题；控制台把这些记录变成可读的项目历史。人类可以短暂离开，再回来像项目经理一样查看任务走向。演讲者用一个生活化例子说明这一点：用户可以异步执行任务，甚至离开去和朋友聚会。这个说法背后的工程含义很硬：系统必须让人类以低频、高质量的方式介入，而无须持续盯着每一步生成。

4.5 异步执行与人在环路中

本章首次定义**异步执行**：它指人类提交目标、批准计划或处理关键干预点之后，任务系统可以在没有持续人工盯守的情况下继续运行，并在需要决策、预算超限、验证失败或范围变化时把控制权交还给人类。异步执行不是放任系统无人负责；它要求系统具备可观察状态、可审计交接和清晰停机条件。

这也补充了人在环路中的工程含义。人在环路中在前文已经出现，本章只强调其运行层面的形态：人类更像项目经理、批准者和边界设定者，负责目标、范围、预算和关键决策；智能体负责在这些边界内执行、验证和交接。人类注意力从“盯每一步”转向“看状态、批计划、处理例外”。

异步执行要求更强的可观察性

任务越异步，控制台、预算、验证和交接越重要。缺少这些机制时，异步只会把风险推迟暴露；具备这些机制时，异步执行才能把人类从持续监督中解放出来，同时保留必要的控制权。

这解释了为什么任务控制台不是装饰性界面。它承担的是运行治理：显示当前活跃智能体在做什么，交接摘要是否可信，预算 B 是否健康，路径是否需要调整。对多日任务而言，界面承担系统运行仪表盘的职责，聊天窗口的美化无法满足这种治理需求。

⁹ 视频画面来源：00:10:29–00:11:21；最终截图取自 00:10:53。

4.6 本章小结

- 全局并行看起来能带来吞吐提升，软件开发中却容易放大协调成本，典型问题包括冲突修改、重复劳动、架构不一致和 token 浪费。
- 串行执行让功能主线单线程推进，降低共享代码状态被多个写入者同时改动的风险。
- 目标内并行允许代码搜索、API 研究、代码审查和验证分析等局部工作并发进行；只读并行是其中风险最低的形式。
- 用 W_f 表示 workflow、 B 表示预算时，长期任务的预算消耗可理解为实现预算、协调预算和返工预算的组合；并行会让后两者快速放大。
- 任务控制台提供状态、预算、交接摘要、活跃智能体和路径调整视图，使人类能以项目经理或批准者身份介入。
- 异步执行要求系统具备可观察状态、可审计交接和清晰停机条件；否则长期任务只是在延后暴露风险。

5 模型路由、专业化智能体与生产案例

前四章已经把任务系统的骨架搭起来：协调器负责规划，工作者负责实现，验证者负责检查；验证合约给出成功条件，结构化交接保存长期上下文，运行策略控制并行与预算。第 5 章继续追问一个更贴近生产的问题：当角色已经分清，系统该把不同环节交给什么模型，又怎样用真实任务数据判断这套架构有没有交付价值。

演讲者在 00:11:22–00:14:32 的核心判断很直接：多智能体架构的质量，不只取决于智能体数量，也取决于角色、模型、验证和企业工作流之间的匹配。规划、执行、验证像三种不同工种：规划像建筑师，需要慢下来推敲依赖关系；执行像熟练工程师，需要快速落地、灵活补齐细节；验证像质检员，需要严格按验收单逐项挑错。让同一个模型包办全部环节，会把系统的上限压在该模型最弱的能力曲线上。

5.1 角色能力差异与模型路由

演讲者首先强调，每个角色都要使用合适的模型。这里首次定义**模型路由**：模型路由指系统按照角色、任务阶段、风险等级和能力曲线，把具体工作分配给适配的模型。它类似交通系统里的分流规则：高速路适合长距离快速通行，支路适合抵达具体地点，安检通道适合发现违禁物；通道本身没有高低之分，关键在于货物、路线和检查目标是否匹配。

与模型路由相连的是**角色化模型选择**：它指工程团队不再只问“哪个模型总体最强”，而是分别问“哪个模型最适合规划，哪个模型最适合执行，哪个模型最适合验证”。规划需要缓慢、仔细的推理，能处理模糊需求、长期依赖和里程碑边界；执行阶段依赖快速代码生成、代码流利度和实现创造力；验证受益于精准指令遵循、稳定检查步骤和独立怀疑视角。

工程判定：模型路由

判定要点：先识别角色、阶段、风险和预算，再选择对应模型。真正有用的路由规则要能解释当前环节为什么交给这个模型，以及失败时由哪个验证环节兜底。

为避免模型路由停留在口号层面，本章用三个符号描述它。令 r 表示角色，常见角色集合为

$$r \in \{\text{plan}, \text{execute}, \text{verify}\}.$$

其中，plan 表示规划角色，对应需求澄清、里程碑设计和验证合约制定；execute 表示执行角色，对应代码实现、集成和修复；verify 表示验证角色，对应审查、测试和端到端检查。再令 L_r 表示分配给角色 r 的模型，则角色化模型选择可以写成

$$r \longmapsto L_r.$$

这个式子的含义很朴素：系统先识别当前工作属于哪个角色 r ，再把它路由给该角色对应的模型 L_r 。如果规划交给善于长链推理的模型，执行交给代码流利的模型，验证交给更会遵循检查指令的模型，系统得到的是复合优势，而非单点模型的平均表现。



图 10: 模型路由表把规划、执行、验证三类角色映射到不同能力曲线。规划侧需要慢而稳的推理，执行侧需要快速代码生成和工具使用，验证侧需要精确遵循指令与独立上下文；因此模型路由可理解为 $L_r = \text{route}(r, K, B)$ ，即按角色 r 、验证合约 K 和预算 B 选择合适模型。¹⁰

还需要一个成功指标来判断路由是否真的有效。令 S 表示成功指标，它可以综合端到端通过率、测试覆盖率、返工次数、成本和耗时。一个轻量的表达是

$$S(r, L_r) = \alpha P_{\text{pass}} + \beta C_{\text{test}} - \gamma N_{\text{rework}} - \delta C_{\text{cost}} - \epsilon T_{\text{latency}}.$$

这里 P_{pass} 表示端到端通过率， C_{test} 表示测试覆盖率， N_{rework} 表示返工次数， C_{cost} 表示 token 或计算成本， T_{latency} 表示耗时； $\alpha, \beta, \gamma, \delta, \epsilon$ 是权重，用来体现项目对质量、成本和速度的偏好。公式表达的工程含义是：某个模型 L_r 对角色 r 的适配度，必须通过可观察结果来判断。验证模型如果便宜却漏掉关键错误， S 会下降；规划模型如果昂贵但显著减少返工， S 可能上升。

类比：能力曲线

模型可以被看成不同形状的工具，而非一把万能刀。规划模型像显微镜，擅长看清依赖和边界；执行模型像电钻，擅长快速打孔和装配；验证模型像游标卡尺，擅长按规格量出偏差。模型路由的目的，是让每种工具出现在适合的位置。

¹⁰概念来源时间区间：00:11:22--00:12:24。

5.2 专业化智能体与机器人驯化

本章首次定义**专业化智能体**：专业化智能体是由角色、工具、提示、模型和验证方式共同塑造的专门执行单元。它比“一个模型实例”更具体，因为同一个底层模型放在不同角色里，会因为提示、工具权限、上下文和验收方式不同而表现出不同工作风格。协调器智能体强调计划与范围控制，工作者智能体强调代码落地，验证者智能体强调挑错和复验。

演讲者把使用这类系统所需的新技能称为“robot whispering”，本章采用**机器人驯化**这个译法。机器人驯化指工程团队理解不同大语言模型的交互方式、强项、失败点，以及这些失败如何在多日任务中累积，然后主动决定哪个模型放在哪个环节、配什么提示、用什么验证方式约束。这个词听起来轻松，背后其实很硬：长期任务中的小偏差会像利息一样滚大，早期一次边界误判，可能在后续功能、测试和迁移里变成成片返工。

风险：失败会累积

多日任务里，模型的小错误很少停留在原地。它可能进入交接摘要，影响后续里程碑，诱导工作者围绕错误假设继续实现。机器人驯化的价值，是提前识别这些模型习性，并用角色边界、验证合约和交接检查把偏差拦住。

机器人驯化还要求团队放弃“默认值永远正确”的幻想。字幕中提到，Factory 内部有人构建了任务原型并设定模型默认值，但团队鼓励根据项目需求自定义这些设置。这个建议很务实：内部工具、重构、迁移、机器学习任务、搜索或研究任务的失败形态并不一样。搜索任务可能需要更强的检索和引用纪律；迁移任务可能更怕遗漏边界条件；前端原型可能更重视交互完整性和速度。专业化智能体的配置，应该跟随任务风险移动。

5.3 模型无关架构

接下来首次定义**模型无关架构**：模型无关架构指任务流程、角色边界、验证合约、里程碑检查点和结构化交接不绑定到单一模型或单一提供商，而是允许不同模型按角色接入、替换和组合。它的核心收益是降低模型供应商锁定，让系统随着模型生态的专业化继续受益。

演讲者给出的判断相当直白：如果锁定单一模型提供商，系统会受限于该家族最弱的能力。某个模型可能写代码很顺，却在长期规划上摇摆；另一个模型可能推理谨慎，却生成代码较慢；还有模型可能非常听指令，适合放在验证环节。模型无关架构把这些差异变成可配置资源。

定义：模型无关架构

模型无关架构把稳定性放在角色、合约、里程碑和交接机制上，而非押注某个模型永远领先。模型可以升级、替换或混用；任务系统的骨架仍然保留规划、执行、验证和状态管理的基本纪律。

验证环节尤其适合体现模型无关架构的优势。字幕中提到，验证环节可以完全使用不同模型提供商，以减少相同训练数据或相同模型家族带来的同源偏差。这里的重点并非“换一个提供商就必然正确”，而是增加独立视角。执行者和验证者如果共享同一套盲点，验证很容易变成复述；引入不同模型家族，相当于让另一种错误分布参与检查。

这种架构还能弥补非前沿模型的不足。验证合约、里程碑检查点和结构化交接像外骨骼：模型自身可能单步能力较弱，但系统把目标拆清楚、把检查点设清楚、把失败反馈写清楚之后，开源模型也能在受控范围内高效推进。模型越强，架构收益越高；模型较弱，架构支撑越关键。

5.4 Slack 克隆案例中的生产数据

演讲者随后把理论拉回生产环境，展示一个 Slack 克隆系统案例。本章首次定义**生产数据**：生产数据指真实任务运行中记录下来的时间、token、验证结果、返工、测试代码、覆盖率、缓存命中和后续功能创建等指标。它的作用像体检报告，能暴露架构在真实压力下的资源分布和质量成本。

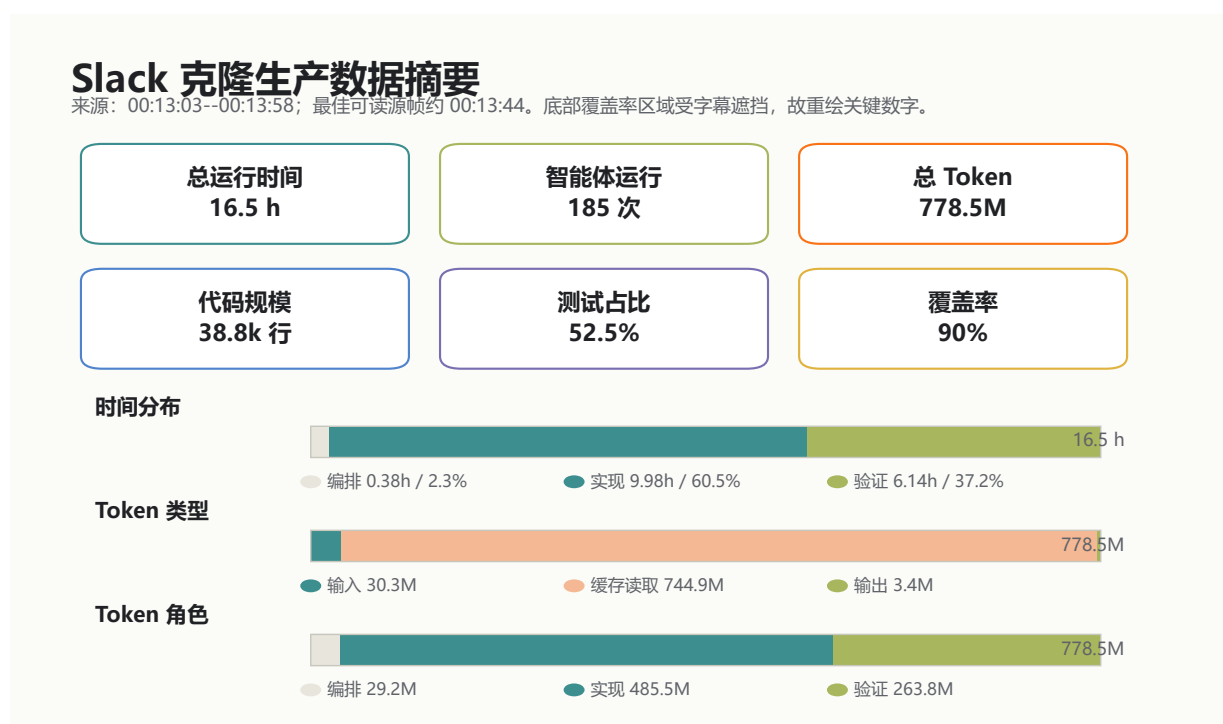


图 11: Slack 克隆生产数据摘要。源帧展示了真实任务的运行时间、智能体运行次数、token 消耗、代码规模、测试占比与覆盖率；重绘图保留可读数字，并补足源画面底部受双语字幕遮挡的覆盖率信息。¹¹

字幕给出的关键数字包括：约 60% 的时间用于实现，同样约 60% 的 token 用于实现；验证在首次尝试时没有直接成功；系统几乎总是需要创建后续功能来修复发现的问题；经过这种循环，最终约 50% 的代码行成为测试代码，约 90% 的代码被测试覆盖；长时间任务的运行成本则通过 Prompt 缓存抵消。

这里首次定义 **QA 循环**：QA 循环指执行、验证、反馈、修复和再验证组成的闭环。它像温控系统：加热器给出动作，温度计发现偏差，控制器调整输出。多智能体任务里，工作者生成候选实现，验证者暴露偏差，后续功能把偏差变成明确修复项，系统再推进到下一轮。

生产信号：QA 循环

Slack 克隆案例显示，首次验证失败不等于任务系统失败。更准确的判断是：系统把过去可能隐藏在人工试用、代码审查或上线后反馈中的质量问题提前显性化，并把它们转化为可追踪的后续功能。

本章也首次定义 **Prompt 缓存**：Prompt 缓存是复用长提示、共享上下文或重复前缀的缓存机制，用来降低长时间任务中的 token 成本和延迟。长期任务会反复携带目标、代码背景、规则、交接摘要和验证条件，如果每轮都从零支付完整上下文成本，预算会迅速膨胀。Prompt 缓存相当于把常用上下文放进仓库货架，后续环节按需取用，减少重复搬运。

测试代码占比约 50%、覆盖率约 90% 这两个数字也值得单独看。很多团队会把测试看成额外负担；在长期智能体任务中，测试更像轨道和护栏。轨道让后续智能体知道哪里可以安全前进，护栏让模型在

¹¹ 视频画面时间区间：00:13:03--00:13:58；最佳可读源帧约 00:13:44，源截图另存为同目录下的生产数据源帧。

上下文切换、修复回归或需求扩展时不至于滑出工程边界。真正的生产交付，必须把质量证据一起交付。

补充：成本结构

长期任务的成本可以粗略分成实现成本、验证成本、修复成本和协调成本。只看实现成本，会低估系统进入生产时的真实代价；忽视验证成本，会把风险转移到用户、线上事故和后续返工中。

5.5 企业 workflows 中的落地场景

演讲者最后把视角扩展到企业场景。本章首次定义**企业 workflow**：企业 workflow 指组织内部围绕业务目标、代码库、权限、审查、数据边界和交付节奏形成的连续工程流程。它远超一次提示词调用，是一串可被计划、执行、验证、审计和移交的工作链。

字幕中列出的场景包括：夜间原型化新想法和功能，更快构建内部工具，执行大规模重构，推进迁移任务，支持机器学习、搜索或研究领域，以及现代化代码库，使智能体在其中更高效。这些任务有一个共同点：它们重要、繁琐、跨文件、跨模块，经常因为人类注意力和排期被拖延。多智能体任务系统适合承担这类目标清晰、细节众多、需要验证、人工盯守昂贵的工作。

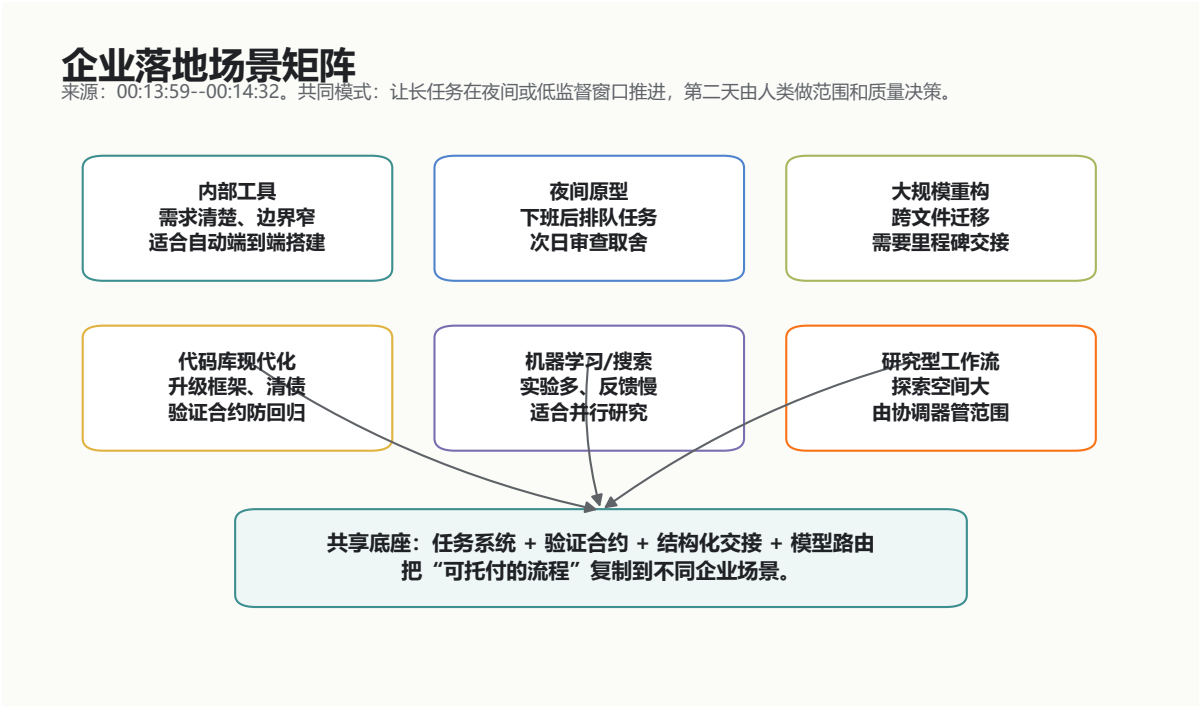


图 12：企业落地场景矩阵。内部工具、夜间原型、大规模重构、代码库现代化、机器学习/搜索和研究型工作流共享同一底座：任务系统负责推进，验证合约定义质量门，结构化交接保存上下文，模型路由匹配角色能力。¹²

场景分类：生成与治理

企业中的智能体任务大致可以分成两类：一类是新能力生成，例如原型、内部工具和新功能；另一类是工程存量治理，例如重构、迁移、测试补齐、依赖升级和代码库现代化。前者扩大产能，后者降低未来每次变更的摩擦。

¹²概念来源时间区间：00:13:59–00:14:32。

企业落地还会遇到一个读者容易低估的问题：智能体越能干，组织越需要清楚地规定边界。权限、数据访问、审查责任、变更窗口、回滚策略和审计记录都会进入 workflow 设计。一个能夜间生成原型的系统，也必须能说明它改了什么、为什么改、验证了什么、还有哪些风险。否则，速度带来的收益会被治理混乱抵消。

因此，第 5 章给出的生产判断可以压缩成一句话：模型路由让不同角色得到合适能力，专业化智能体让能力进入可管理岗位，模型无关架构让系统摆脱单一模型家族的短板，生产数据和 QA 循环则负责证明这些选择是否真的能交付。企业 workflow 是最终考场；只有在真实代码库、真实预算和真实质量门里站住，架构才算落地。

5.6 本章小结

- 模型路由按照角色、阶段和风险选择模型；角色化模型选择用 $r \mapsto L_r$ 表达角色到模型的分配。
- 成功指标 S 应同时观察端到端通过率、测试覆盖率、返工次数、成本和耗时，避免只凭模型名或单次演示判断效果。
- 专业化智能体由角色、工具、提示、模型和验证方式共同塑造；机器人驯化要求理解模型失败点及其多日累积效应。
- 模型无关架构通过验证合约、里程碑检查点和结构化交接降低单一提供商绑定，并允许不同模型家族分别承担规划、执行和验证。
- Slack 克隆案例显示，生产任务的资源大量分布在实现、验证、修复、测试和缓存管理中；QA 循环是质量进入生产的关键路径。
- Prompt 缓存用于抵消长时间任务的上下文成本；企业 workflow 则把智能体任务放进权限、审查、审计、迁移和代码库现代化等真实组织约束中。

6 持续学习、软件工程经济学与开放问题

第 5 章已经说明，这套多智能体架构能够进入真实生产：模型路由让规划、执行、验证分别获得合适能力，专业化智能体把能力安放到明确岗位，生产案例则暴露出实现、验证、返工、测试和缓存的真实成本。第 6 章把问题推到更长周期：模型还会继续变强，企业流程还会继续变复杂，架构怎样避免在下一代模型发布时立刻过时，又怎样把第 1 章提出的人类注意力瓶颈真正移走。

演讲者在 00:14:34–00:18:14 的结尾并没有把多智能体系统讲成一次性技巧。他强调的是一种可演进的工程观：任务系统提供纪律、记账、停机和验证；模型提供基础智能；人类则从盯每一步执行，转向定义目标、批准计划、设计系统契约和判断产品方向。这个转向，是全文从协作模式走向可落地交付的最后一环。

6.1 苦涩教训与持续学习

本章首先定义**持续学习**：持续学习指系统随着底层模型进步、提示改进、技能积累和反馈循环不断优化自身执行效果。它的重点不在于让某个单次对话“记住更多”，重点在于让任务分解、验证、交接、模型路由和技能配置都能被生产反馈持续校准。类比软件团队的工程体系，持续学习像把复盘、测试、模板、规范和工具链更新沉淀下来，让下一次任务天然站在上一次任务的结果之上。

演讲者用**苦涩教训**解释这种设计取向。苦涩教训来自人工智能史上的一个朴素事实：长期来看，能够利用更多计算和更强通用方法的系统，通常会超过依赖大量人工规则和手工特例的系统。在多智能

体架构中，它意味着系统应当让底层模型能力的提升自然转化为任务收益。如果新模型更会规划、更会写代码、更会遵循验证步骤，架构应该立刻吃到这部分红利，而不需要把核心流程大改一遍。

工程含义：持续学习与苦涩教训

判定要点：模型升级、提示迭代、技能沉淀和验证反馈都要进入系统主干。若生产反馈只被记录下来，却没有改变提示、技能、合约或路由策略，这套系统就没有真正吸收经验。

这也是第 5 章模型无关架构的自然延伸。模型路由解决“哪个角色用哪个模型”的问题，持续学习解决“模型变强之后系统如何同步变强”的问题。两者合起来，才构成可长期维护的任务系统：前者像插座标准，让不同设备接入；后者像电网升级，让更强设备接入后能释放功率。

用一个轻量公式表示，若任务系统在时间 t 的能力为

$$A(t) = f(M(t), P(t), K, \mathcal{H}, R(t)),$$

其中 $M(t)$ 表示可用模型能力， $P(t)$ 表示提示与技能配置， K 表示验证合约， $\mathcal{H}$ 表示结构化交接， $R(t)$ 表示从生产任务中得到的反馈，那么持续学习要求

$$\frac{\partial A}{\partial M} > 0, \quad \frac{\partial A}{\partial R} > 0.$$

这个表达并不追求严格度量，它强调两个方向：模型进步应该增加系统能力，反馈积累也应该增加系统能力。如果模型变强但系统效果不变，说明架构把智能锁在了流程外；如果反馈很多却不改变提示、技能和验证策略，说明系统只是在记录失败，并未吸收失败。

6.2 提示/技能驱动编排与硬编码状态机

本章定义**提示/技能驱动编排**：它指把任务分解、失败处理、执行策略和角色行为主要放进可阅读、可修改、可版本化的提示与技能中，由模型在这些约束下完成具体推理和行动。这里的**编排**是系统级流程设计，关注目标如何被拆成计划、里程碑、角色调用、验证和交接。第 2 章的协调器是编排中的一个关键角色，第 6 章讨论的是编排规则应该放在哪里。

与之相对的是**硬编码状态机**。硬编码状态机指开发者把流程分支、失败恢复、重试策略和下一步选择写死在程序逻辑里，让系统按固定状态跳转运行。状态机本身在支付、编译器、协议实现等场景很有价值，因为那些场景需要确定性和可证明边界；但在多智能体任务系统里，需求模糊、失败形态多、模型能力快速变化，过度硬编码会让系统像被焊死的机械臂，只能重复旧动作。

提示/技能驱动编排 vs 硬编码状态机

来源：00:14:34--00:15:50。策略放在可改写的提示与技能中，模型升级时系统能继续吸收新能力。



图 13: 提示/技能驱动编排与硬编码状态机的差异。前者把任务分解、失败处理和策略演进放进可调整的提示与技能，让模型能力进步能够转化为系统收益；后者把大量策略判断固定在分支和转移表中，模型升级后仍会受到旧流程约束。¹³

类比：菜谱与轨道

提示/技能驱动编排像可改写的菜谱：主厨能力提升后，同一份菜谱可以做得更好，菜谱本身也能根据试吃反馈调整。硬编码状态机像固定轨道：列车运行稳定，但新路线、新岔路和临时绕行都需要改造轨道。

字幕中提到，Factory 把几乎所有编排逻辑定义在提示和技能中，包括如何分解失败或功能、如何处理故障、如何显著改变执行策略。工作者行为由编排器为每个任务定义的技能驱动，因此可以产生高度定制化行为。确定性逻辑则保持很薄，重点放在让模型发挥最佳能力，以及让系统承担账目管理式工作：执行验证、记录状态、在未解决交接问题存在时阻止继续推进。

定义：提示/技能驱动编排

提示/技能驱动编排把流程策略放进可迭代的提示和技能，把固定程序逻辑收缩到验证、状态、权限、预算和停机等基础控制面。它的目标是让模型承担开放推理，让系统保留工程纪律。

这一区分也解释了“纪律性与智能”的分工。纪律性来自任务系统：验证合约规定成功条件，结构化交接保留上下文，任务控制台呈现预算和状态，里程碑边界拦截错误。智能来自模型：它理解目标，生成代码，处理模糊需求，尝试修复失败。两者关系像机场运行：塔台负责规则、跑道、时隙和安全检查；飞行员负责驾驶、判断天气和应对动态情况。塔台不能替代飞行员驾驶，飞行员也不能随意取消塔台规则。

¹³概念来源时间区间：00:14:34–00:15:50。

6.3 软件工程经济学的变化

本章定义**软件工程经济学**：它指软件团队在时间、注意力、计算成本、返工风险、质量收益和组织协作之间做取舍的成本结构。传统工程经济学中，瓶颈经常落在人类工程师的排期、审查、上下文切换和执行时间上；多智能体任务系统改变的是这张账单的分布。

演讲者回扣第 1 章的人类注意力瓶颈：一个五人工程团队过去也许只能同时推进十个工作流，现在通过任务系统有机会提升到三十个。这个数字不应被理解成机械的三倍产能承诺，它更像一个方向信号：人类注意力从逐行盯执行，迁移到挑选目标、判断范围、批准计划、设定验证合约、处理例外和做产品取舍。



图 14：注意力瓶颈从执行监督迁移到系统设计。任务系统吸收排队任务、上下文切换、逐项审查、故障追踪和返工沟通等低层监督负担，使人类判断更多集中在架构设计、产品决策、边界设定、验证契约和开放问题上。¹⁴

风险：吞吐提升会转移瓶颈

任务系统降低了执行盯守成本，但不会自动消除产品判断、架构责任和组织治理。若团队只增加任务数量，却没有加强范围审查、验证合约和交接质量，新的瓶颈会出现在错误任务被高速执行之后。

这里需要引入**系统设计杠杆**：系统设计杠杆指少数架构选择对大量后续任务产生的放大效应。验证合约就是杠杆，因为它一次定义质量条件，后续执行、验证和返工都围绕它展开；结构化交接也是杠杆，因为它把长期任务的记忆从对话流转移到可检查记录；模型路由同样是杠杆，因为角色与模型的匹配会影响每个里程碑的质量和成本。

如果把团队总产出粗略写成

$$O \approx \frac{N_{\text{task}} \cdot Q_{\text{avg}}}{C_{\text{attention}} + C_{\text{coordination}} + C_{\text{rework}}},$$

¹⁴概念来源时间区间：00:15:50–00:16:31。

其中 N_{task} 是可推进任务数, Q_{avg} 是平均质量, $C_{\text{attention}}$ 是人类注意力成本, $C_{\text{coordination}}$ 是协调成本, C_{rework} 是返工成本, 那么多智能体系统的目标并非单纯增大 N_{task} 。它还要通过验证、交接和运行策略压低返工与协调成本, 并把人类注意力投入到高杠杆环节。

定义：软件工程经济学与系统设计杠杆

软件工程经济学观察软件交付中的成本、质量、速度和注意力分配。系统设计杠杆则指那些一经设计好就能反复放大收益的结构, 例如验证合约、结构化交接、模型路由和任务控制台。

第 5 章的生产案例让这个经济学判断更具体。Slack 克隆案例中, 验证首次尝试经常失败, 后续功能会被不断创建, 测试代码和覆盖率占据很高比例。这些数字说明, 真实价值不在于“模型一次生成完美代码”, 而在于系统把质量问题提前显性化, 把返工转化为有边界的后续任务, 把长期成本放进可观察账本。第 3 章的验证合约与第 5 章的 QA 循环在这里闭合: 前者定义什么叫做成功, 后者把失败变成可执行修复。

6.4 从原始策略到可落地系统

演讲者在结尾回看全文时, 强调任务系统是原始协作策略的组合。第 1 章的五类协作模式提供了积木: 委托让协调器生成工作者或研究子智能体; 创建者-验证者对让实现和检查保持分离; 广播把任务状态传递给相关角色; 协商发生在里程碑边界、范围判断和交接质量检查中; 单一事实来源则避免长期任务被散落对话拖偏。

这些积木在第 2 章被组装成三角任务系统, 在第 3 章通过验证合约和结构化交接获得正确性与记忆, 在第 4 章通过串行主线和目标内并行降低协调成本, 在第 5 章通过模型路由和生产案例进入真实工程压力。第 6 章补上的, 是持续演进机制: 当模型变强、技能更新、反馈增多, 架构仍能沿着同一套角色、合约和交接机制继续改进。

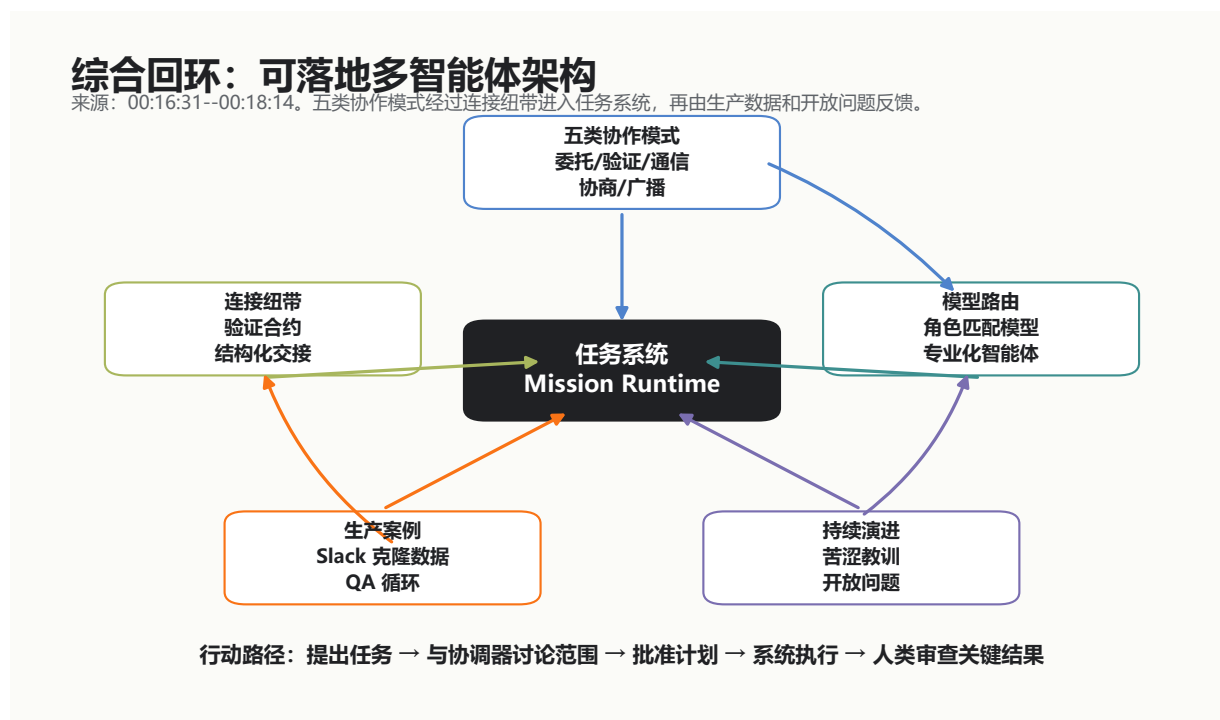


图 15: 综合回环图把全文主线压缩为一张架构图: 五类协作模式进入任务系统, 验证合约和结构化交接提供连接纽带, 模型路由匹配专业化智能体, 生产案例与开放问题再反馈到下一轮编排设计。¹⁵

全文闭环：从注意力瓶颈到架构循环

全文的闭环可以压缩为一条链：人类注意力瓶颈提出问题，协作模式提供积木，任务系统组织角色，验证合约定义成功，结构化交接保存记忆，运行策略控制冲突，模型路由匹配能力，生产数据反馈改进，持续学习让下一轮任务更强。

这条链有一个容易被忽略的含义：多智能体架构的核心，在于让目标、状态、责任、验证和反馈拥有稳定位置，而非单纯让更多智能体同时说话。智能体数量增加只会放大系统原本的结构质量。结构清楚时，更多智能体意味着更多可控工作；结构混乱时，更多智能体会制造更多上下文碎片、重复劳动和返工。

6.5 开放问题

本章定义**开放问题**：开放问题指当前架构已经显示有效性，但仍缺少稳定答案、成熟实践或充分生产数据的问题。开放问题更像研究路线图，而非失败声明。它提醒读者，真实系统已经能在大规模项目中产生价值，同时仍有大量工程细节等待被压缩、标准化和验证。

演讲者提出的第一类开放问题，是怎样进一步减轻任务工作量，让任务运行得更快。这里的“工作量”既包括模型调用和 token 消耗，也包括验证等待、环境启动、交接阅读、失败定位和后续功能生成。降低工作量可以来自更好的任务分解、更精准的验证合约、更强的缓存、更快的用户测试验证器，也可以来自更好的技能，让工作者少走弯路。

第二类开放问题，是怎样把任务编排到更复杂的流程中。单个任务系统已经能处理多日软件工程目标，但企业真实流程还包括权限审批、合规审计、发布窗口、数据迁移、跨团队依赖、事故回滚和长期维护。复杂流程像城市交通，单辆车会开还不够，还需要红绿灯、道路规则、施工提示、事故处理和公共调度。

开放问题：规模化之后的治理

任务系统越能自主推进，越需要清楚回答谁批准范围、谁拥有变更责任、谁检查验证合约、谁处理失败升级、谁决定任务停止。否则，自动化会把模糊责任放大成组织风险。

第三类开放问题，是怎样培养能理解模型组合的人。演讲者特别提到，真正推动下一代创新的人，往往是那些理解智能体生态、理解不同模型在压力下如何组合的人。这已经超出“会写提示词”的层面，更接近工程经理、架构师、质量负责人和工具平台团队的交叉能力：他们要懂模型能力曲线，也要懂代码库、测试、交付节奏和组织约束。

6.6 本章小结

- 持续学习让任务系统随着模型进步、提示改进、技能积累和生产反馈持续优化。
- 苦涩教训要求架构把通用模型能力和计算增长接入主干，避免把短期经验冻结为僵硬流程。
- 提示/技能驱动编排把任务分解、失败处理和执行策略放进可迭代文本与技能中；硬编码状态机适合确定性强的局部控制，但不宜承担全部开放任务逻辑。
- 软件工程经济学的变化在于，人类注意力从具体执行迁移到架构设计、产品决策、验证合约和系统治理。

¹⁵概念来源时间区间：00:16:31–00:18:14。

- 系统设计杠杆包括验证合约、结构化交接、模型路由、任务控制台和持续反馈循环，它们会反复影响后续任务质量与成本。
- 开放问题集中在减轻任务工作量、编排复杂企业流程、治理自主任务责任，以及培养理解模型组合和工程约束的人才。

6.7 最终收束：把执行交给系统，把判断留给人

演讲者的实质性结尾很克制：尝试运行任务，与编排器讨论范围，批准计划，然后去做其他事；他期待看到听众构建出的成果，并把剩余时间留给问答。这个结尾没有炫技姿态，落点是全文主张的实践方式。系统的价值，在于让人类不必持续盯着每一次文件搜索、每一次测试运行、每一次修复尝试，却仍能通过计划、验证合约、交接和控制台掌握任务方向。

全文核心主张可以压缩成一句话：可落地的多智能体架构，是把智能体协作从聊天流变成有角色、有合约、有记忆、有验证、有预算、有反馈的工程系统。第 1 章的人类注意力瓶颈提出了为什么需要它；第 3 章的验证合约说明了怎样判断做对；第 5 章的生产案例证明了质量问题、返工和测试可以被系统化管理；第 6 章则说明，这套结构必须持续学习，才能在模型快速进步时保持生命力。

对实践者而言，行动建议很明确。第一，从一个真实但边界可控的任务开始，不要用玩具问题误判系统价值。第二，在动手前和编排器讨论范围，让模糊需求先变成计划、里程碑和验证条件。第三，批准计划之后，让任务系统执行，并通过任务控制台查看状态、预算、交接和验证结果。第四，把失败当成系统输入：验证失败、交接缺口、范围误判和成本异常，都应该沉淀为提示、技能、合约和路由策略的改进。

开放问题仍然摆在前面：任务怎样更轻、更快、更便宜；复杂企业流程怎样被安全编排；模型组合怎样在压力下保持稳定；人类团队怎样重新分配责任和注意力。演讲者给出的方向很清楚：未来的软件工程团队不会只比较谁写代码更快，而会比较谁更会设计能持续学习的交付系统。真正的杠杆，落在目标、契约、验证、交接、模型路由和反馈循环这些结构上。代码仍然重要，但代码已经进入一条更大的生产线；人类最稀缺的资源，也从亲自执行每一步，转向判断哪一步值得执行、怎样验收、何时停止、下一轮如何变得更好。